# SMART LIBRARY MANAGEMENT SYSTEM

# Z Notation Formal Specification

## 1. Introduction:

The Smart Library Management System is an automated solution designed to efficiently manage library operations using formal methods, specifically **Z Notation**.

This system ensures accuracy, consistency, and validation of operations such as issuing and returning books, enforcing borrowing limits, managing fines, and handling notifications.

## 2. Core Features:

The system includes the following key functionalities:

- Book Issuing
- Book Returning
- Book Availability Checking
- Maximum 3 Books Constraint
- One Book Assigned to One Student
- Fine Calculation and Online Payment
- Barcode Scanning for Book Identification
- Notifications and Reminders

## 3. System State Schema:

### Z Specification:

```
LibrarySystem
books : ℙ BOOK
students : ℙ STUDENT
issued : BOOK ↔ STUDENT
fines : STUDENT → ℕ
```

### Explanation:

This schema defines the overall state of the system:

- **books** represents the set of all available books
- **students** represents registered users
- **issued** is a relation mapping books to students
- **fines** stores the fine amount associated with each student

## Screenshot:

```
[BOOK, STUDENT]

LibrarySystem
books : ℙ BOOK
students : ℙ STUDENT
issued : BOOK ↔ STUDENT
fines : STUDENT ⇸ ℕ



------------------------------------------------
```

# 4. Initialization Schema:

## Z Specification:

```
InitLibrary
LibrarySystem
books = ∅
students = ∅
issued = ∅
fines = ∅
```

## Explanation:

This schema initializes the system with an empty state:

- No books are issued
- No students are registered
- No fines are recorded

## Screenshot:

```
------------------------------------------------
∀ s : STUDENT • #(issued~[{s}]) ≤ 3
∀ b : BOOK • #(issued[{b}]) ≤ 1
InitLibrary
LibrarySystem
```

# 5. Issue Book Operation:

## Z Specification:

```
IssueBook

ΔLibrarySystem
s? : STUDENT
b? : BOOK

b? ∈ books
b? ∉ dom issued
#(issued ▷ {s?}) < 3

issued' = issued ∪ {b? ↦ s?}
```

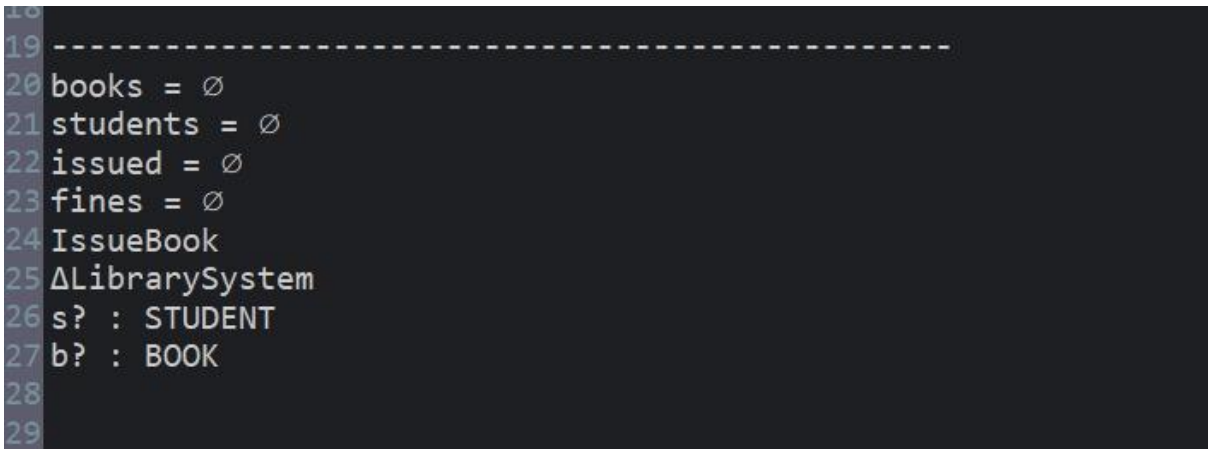
## Explanation:

This operation allows a student to issue a book under the following conditions:

- The book must exist in the library
- The book must not already be issued
- The student must have fewer than 3 books issued

The system updates the issued relation accordingly.

## Screenshot:



```
----------------------------------------------
books = ∅
students = ∅
issued = ∅
fines = ∅
IssueBook
ΔLibrarySystem
s? : STUDENT
b? : BOOK
```

# 6. Return Book Operation:

## Z Specification:

```
ReturnBook
ΔLibrarySystem
b? : BOOK

b? ∈ dom issued

issued' = issued \ {b?}
```

## Explanation:

This operation allows a book to be returned:

- The book must already be issued
- The system removes the book from the issued relation

## Screenshot:

```
----------------------------------------------
s? ∈ students
b? ∈ books
b? ∉ dom issued
#(issued~[{s?}]) < 3

issued' = issued ∪ {b? ↦ s?}
books' = books
students' = students
fines' = fines
ReturnBook
ΔLibrarySystem
b? : BOOK
```

# 7. Maximum 3 Books Constraint:

## Z Specification:

```
MaxBooks
LibrarySystem
∀ s: STUDENT • #(issued ▷ {s}) ≤ 3
```

## Explanation:

This constraint ensures that:

- Each student can hold a maximum of **three books at a time**

## Screenshot:

```
---------------------------------------------
b? ∈ dom issued

issued' = {b?} ⩤ issued
books' = books
students' = students
fines' = fines
MaxBooks
LibrarySystem
```

# 8. One Book → One Student Constraint:

## Z Specification:

```
UniqueIssue
LibrarySystem
∀ b: BOOK • #(issued[{b}]) ≤ 1
```

## Explanation:

This rule ensures that:

- A single book can only be issued to **one student at any given time**

## Screenshot:

```
---------------------------------------------
∀ s : STUDENT • #(issued~[{s}]) ≤ 3
UniqueIssue
LibrarySystem
```

# 9. Fine Payment Operation:

## Z Specification:

```
PayFine
```

```
ΔLibrarySystem
s? : STUDENT
amount? : ℕ

fines(s?) ≥ amount?

fines' = fines ⊕ {s? ↦ fines(s?) - amount?}
```

## Explanation:

This operation handles fine payments:

- The student must have sufficient outstanding fine
- The system deducts the paid amount from the total fine

## Screenshot:

```
66 ----------------------------------------------
67 ∀ b : BOOK • #(issued[{b}]) ≤ 1
68 PayFine
69 ΔLibrarySystem
70 s? : STUDENT
71 amount? : ℕ
72
73
74
```

# 10. Barcode Scanning:

## Z Specification:

```
ScanBook
ΞLibrarySystem
b? : BOOK

b? ∈ books
```

## Explanation:

This schema validates scanned books:

- Ensures that the scanned book exists in the system
- No state change occurs (read-only operation)

## Screenshot:

```
---------------------------------------------
s? ∈ dom fines
fines(s?) ≥ amount?

fines' = fines ⊕ {s? ↦ fines(s?) - amount?}

books' = books
students' = students
issued' = issued
ScanBook
ΞLibrarySystem
b? : BOOK
```

# 11. Notification System:

## Z Specification:

```
Notify
ELibrarySystem
s? : STUDENT

s? ∈ students
```

## Explanation:

This schema represents system notifications:

- Sends reminders to registered students
- Used for due dates and alerts
- Does not modify system state

## Screenshot:

```
---------------------------------------------
b? ∈ books
Notify
ΞLibrarySystem
s? : STUDENT

s? ∈ students

```

# 12. Conclusion:

The Smart Library Management System has been successfully modeled using Z Notation.

All system operations and constraints have been formally specified, ensuring:

- Logical correctness
- Data consistency
- Constraint enforcement

This formal specification serves as a strong foundation for system validation and verification.